



RUNTIME FEEDDOWNLOADER PRO

Advanced User Manual

AUGUST 2026

ECOSYSTEM RUNTIME | DIGITAL CORE

Chapter 1: Introduction and Philosophy

1.1 What is Runtime FeedDownloader Pro?

To describe the software, it is useful to begin with the problem it solves.

Every day, thousands of podcast episodes are published, distributed and listened to. Over time, however, a significant portion of this content disappears: the host stops paying for their hosting service, the distribution platform ceases operations, or the CDN serving the audio files is decommissioned. An episode listened to three years ago may today be permanently unreachable — not because it was intentionally deleted, but because no one retained a copy.

Runtime FeedDownloader Pro was created to address this problem. It is not a simple podcast download tool: it is a professional application for the **systematic preservation and archiving** of audio content from RSS feeds. It is designed for archivists, publishers, radio stations, content producers and enthusiasts for whom sound documentation requires the same rigorous preservation standards applied to other types of records.

1.2 Who It Is For

FeedDownloader Pro addresses a range of different needs:

- **The Archivist:** Wants to download the entire catalogue of a historical podcast before it is removed. Needs a system that remembers previously downloaded episodes, avoids duplicates and verifies the integrity of every file.
 - **The Radio Producer:** Manages a content library on a shared NAS. Needs a tool that operates on network paths without stalling, organises files predictably and produces CSV reports for their team.
 - **The Publisher:** Wants to maintain a local copy of all podcasts in their network, export metadata for content management systems and monitor the archive's status over time.
 - **The Enthusiast:** Wants to keep their favourite podcasts on their disk, organised neatly, without depending on internet availability or risking receipt of corrupted files.
-

1.3 The "Database-First" Philosophy

The fundamental difference between FeedDownloader Pro and a generic download tool is its approach to data management.

Most download tools work as follows: they analyse the files present on disk, compare them with the RSS feed and download whatever is missing. This approach has a critical limitation: **the disk is not a reliable source of truth**. Files can be moved, renamed, corrupted or accidentally deleted. If you move the podcast folder from `C:\Podcast` to `D:\Archive`, the tool loses its reference to previously downloaded episodes and begins downloading the entire catalogue again.

FeedDownloader Pro takes a different approach. At the centre of every operation is a **SQLite database** that records every episode analysed or downloaded: the original URL, the file path on disk, the download date, the SHA-256 hash of the content and the audio metadata. The database is the software's persistent memory. Regardless of the physical location of the files, the database preserves the complete state of the archive.

This architecture has direct practical consequences:

1. **No duplicates.** Even if the same feed is analysed multiple times, the system recognises episodes already present in the database and does not add them to the queue again.
 2. **Resilience to moves.** The archive can be moved to a new disk or to a NAS: the history remains intact in the database.
 3. **Persistent state between sessions.** If the program is closed during a batch download of 300 episodes, the queue is available in exactly the same state when it is reopened.
 4. **Operation log.** Every downloaded file is documented: download date, source URL and integrity verification status.
-

1.4 The Three Pillars of the Software

Beyond the Database-First approach, FeedDownloader Pro is built around three technical principles with a direct impact on functionality.

Network Resilience

Downloading hundreds of audio files sequentially over the internet is not a straightforward operation. Servers can be overloaded, connections

can drop and transfers can corrupt files. FeedDownloader Pro handles these scenarios with three mechanisms:

- **Retry with exponential backoff:** When a download fails, the software does not retry immediately. Instead, it waits for a progressively longer interval: 1 second, then 2, then 4. This approach, standard in distributed systems, increases the probability of success without placing additional load on the source server.
- **Stall detection:** A stalled download is more problematic than a failed one. If a server begins sending data and then stops without closing the connection, software without this check would wait indefinitely. FeedDownloader Pro monitors the data flow in real time: if no new bytes arrive for 60 consecutive seconds, the download is terminated and automatically re-queued.
- **Anti-corruption .part files with resume:** Every file is downloaded with the temporary extension `.part`. Only upon total, verified completion of the transfer is the file renamed with its final extension (`.mp3`, `.m4a`, etc.). In the event of an interruption, the `.part` file is kept: on the next attempt the download **resumes from where it stopped** (HTTP Range request with an If-Range validator) instead of starting over. Orphaned `.part` files can be removed via **Settings** → **Advanced** → **Clean temporary files**.

Built-in Security

FeedDownloader Pro processes URLs from external sources (RSS feeds). A maliciously constructed URL pointing to internal network resources (a router, a NAS, a local server) could be used to access confidential information — an attack known as **SSRF (Server-Side Request Forgery)**.

To prevent this risk, every URL is subjected to a **multi-level validation** process before being processed: syntactic checks, acceptance of

HTTP/HTTPS protocols only, blocking of well-known internal host-names, blocking of private and reserved IP ranges (RFC 1918, loopback, link-local), and re-validation of the DNS-resolved IP address on every connection and every redirect. This procedure is entirely automatic and transparent to the user.

NAS and Network Path Support

FeedDownloader Pro is designed to operate with archives on network drives. Handling SMB paths — the protocol used by NAS devices, Windows servers and network shares — is a frequent source of problems in desktop applications: an unreachable network drive can block the application's main thread for a considerable amount of time. FeedDownloader Pro resolves this by running network path validation on a separate thread with an 8-second timeout. The interface remains responsive at all times, regardless of the state of the network path.

1.5 Contents of This Manual

This manual covers the complete use of FeedDownloader Pro, from installation to the most advanced features. It need not be read in sequence: each chapter is self-contained and may be consulted independently.

For a first approach to the software, it is recommended to follow **Chapter 4 (Your First Archive)**, which illustrates a complete workflow from feed analysis to download. Those already familiar with the software may navigate directly to the chapter of interest via the table of contents.

Ecosystem Runtime | Digital Core – Tools built to last.

Chapter 2: Installation and First Launch

2.1 System Requirements

Runtime FeedDownloader Pro is a desktop application based on Electron technology. It is self-contained and does not require the installation of additional runtimes (Node.js, .NET, Java): everything necessary is included in the installation package.

Minimum requirements:

Operating System	Minimum Version	Architecture
Windows	10 (build 1903) or Windows 11	64-bit (x64)
Linux	Ubuntu 22.04 LTS, Debian 11, Fedora 36 or equivalent distributions	64-bit (x64)

Recommended hardware:

- **RAM:** 4 GB (8 GB recommended for large archives with multiple active threads)
- **Disk space:** 200 MB for the program installation, plus the space required for the audio archive
- **Connection:** Broadband (at least 10 Mbps to use parallel downloads effectively)

Note for Linux users: The software is distributed in `.AppImage` format (self-contained, usable on any modern distribution with up-to-date glibc libraries, without a traditional installation procedure) and in `.deb` format for Debian/Ubuntu-based distributions.

Note on macOS: Official macOS builds are not distributed. The code is compatible with Electron on macOS, but compiling it is left to the user and is not supported.

2.2 Installation on Windows

1. Download the installation file `Runtime-FeedDownloader-Pro-1.5.0.exe` from the official releases page.
2. Double-click the downloaded file to launch the installer.
3. If Windows displays a "**Windows protected your PC**" warning (SmartScreen), click "**More info**" and then "**Run anyway**". This warning is standard for software distributed outside the Microsoft Store that has not yet reached a sufficient adoption threshold for Windows' reputation system.
4. Follow the on-screen instructions: accept the licence agreement, choose the installation folder and click "**Install**".
5. Upon completion, a shortcut will be available on the **Desktop** and an entry in the **Start** menu.

Installation and data paths: The program is installed in `C:\Program Files\Runtime FeedDownloader Pro\`. The database is saved separately in `C:\Users\[YourName]\AppData\Roaming\Runtime FeedDownloader Pro\`. This separation ensures that uninstalling the program does not affect the archive data.

2.3 Installation on Linux

1. Download the file `Runtime-FeedDownloader-Pro-1.5.0.AppImage` (or the `.deb` package for Debian/Ubuntu).
2. Make the `.AppImage` file executable. Available methods are:
 - **Via graphical interface:** Right-click the file → Properties → Permissions tab → tick "Allow executing file as program".
 - **Via terminal:** `chmod +x Runtime-FeedDownloader-Pro-1.5.0.AppImage`
3. Launch the file with a double-click or from the terminal: `./Runtime-FeedDownloader-Pro-1.5.0.AppImage`

For the `.deb` package: install with `sudo dpkg -i Runtime-FeedDownloader-Pro-1.5.0.deb` (or with your distribution's graphical package manager).

Desktop integration (optional): To add FeedDownloader Pro to the launcher and application menu, use **AppImageLauncher** (available in the repositories of most distributions), which automatically integrates AppImage files into the system.

Note for sandboxed environments: On distributions using **Flatpak** or environments with filesystem access restrictions, the software may not be able to reach SMB network paths. In this case, verify that the network filesystem is mounted and accessible from the file manager before launching the program.

2.4 First Launch

On first opening, the software is immediately ready to use. No initial configuration is required, nor is it necessary to create an account or enter a licence key. The interface presents the URL input bar at the centre and an empty episode list.

Files created on first launch: The program automatically generates the following file in the user data folder:

- `feeddownloader.sqlite` — The main SQLite database. Contains the complete download history, episode metadata, user preferences (language, parallel downloads, destination folder, etc.) and archive status. **This file must not be deleted.**
-

2.5 Updates

The update system always works **with the user's consent**, in three steps:

1. On launch (and on demand via **Settings** → **Advanced** → **Check for Updates**), the software checks whether a new version exists. If one is available, a persistent **"Update available"** indicator appears in the top bar of the interface.
2. The package download starts **only** when you press the indicator's **"Download"** button (or **"Download update"** in the Settings). No download ever starts automatically.
3. Once downloaded, the indicator changes to **"Update ready"**: press **"Restart & install"** to apply the update. Installation, too, never happens automatically.

Archive data is not modified during an update: only the program files are replaced. On the first launch after an update, the **"What's new in this version"** window summarises the changes introduced.

Note: Before updating to a major version (for example from 1.4.x to 1.5.x), it is advisable to make a manual copy of the `feeddownloader.sqlite` file in a safe location.

Go to Chapter 3 for a detailed description of the interface elements.

Chapter 3: Interface Tour

3.1 Anatomy of the Main Window

When FeedDownloader Pro opens, the window is organised into four functional zones:

- **Command bar (top):** The fixed bar containing the URL field, the Analyse button, the Command Palette button and the Settings icon. When a software update is available, the persistent update indicator also appears in this bar (see Chapter 2, section 2.5). All operations for adding new feeds are initiated from here.
- **Feed sidebar (left):** The column containing the permanent library of saved feeds, the Archive tab, synchronisation controls, and the footer showing the destination path. Its width can be adjusted by dragging the right edge.
- **Main area (centre):** The area where the episodes of the selected feed are displayed, with the filter bar, batch controls, and the episode list.
- **Download Panel (right, overlaid):** The panel that opens automatically when a download batch is in progress. When closed, a floating button remains visible at the bottom right to reopen it.

On first launch, with an empty library, the main area shows a guide message explaining how to add the first feed and where to configure the destination path. The message disappears automatically when the first feed is added, or manually via the × button.

3.2 The Command Bar (Top)

URL Field: The text bar where you enter the RSS address of the podcast to analyse. Accepts direct URLs to XML/RSS files. Supports **drag and drop**: you can drag a link directly from a browser onto this area.

"Analyse" Button: Starts the feed analysis. The software contacts the URL, reads the RSS file, and populates the episode list. On completion of the analysis, the feed is permanently added to the sidebar. The operation generally takes between 1 and 5 seconds, depending on the feed size and connection speed.

Settings Icon (⚙): Opens the settings panel. It is accessible at any time, even during an active download. For details, see Chapter 10.

3.3 The Feed Sidebar

The sidebar is the control centre of the podcast library. It contains all added feeds permanently: feeds are not lost when the software is closed.

The Feed and Archive Tabs

At the top of the sidebar there are two tabs:

- **Feed Tab:** Shows the library of saved feeds. This is the default view.
- **Archive Tab:** Shows the Archive View — a table of all episodes downloaded across the entire library. See section 3.10.

Feed library items

Each feed in the list is represented by a row showing:

- **Thumbnail:** The podcast cover image.
- **Title:** The name of the podcast as declared in the RSS feed.
- **Date:** The date of the last synchronisation with the server.
- **"TO DOWNLOAD" badge:** An indicator (e.g. **3 TO DOWNLOAD**) showing how many new episodes have been detected since the last synchronisation. New episodes are recognised via each episode's unique identifier (GUID), recorded in the database: an episode counts as new only if its GUID has never been seen before. The badge disappears after all new episodes have been downloaded.

Clicking on a list item loads that feed's episode list in the main area.

Adding a feed

Paste the RSS URL into the URL field at the top of the interface and click "Analyse". Once the analysis is complete, the feed is automatically added to the sidebar and remains available for subsequent sessions.

Search and sorting

- **Feed search:** The search field in the sidebar filters feeds by name in real time. Useful with large libraries.
- **A-Z sorting:** The sort button orders feeds alphabetically by title. Clicking it again restores the original order.

Synchronisation

- **Individual synchronisation:** Hovering over a feed item reveals the synchronisation icon. Clicking it causes the software to re-read the

feed from the server and update the episode list with any new content.

- **Sync All:** The "Sync All" button at the top of the sidebar updates all feeds in parallel. During the operation, each thumbnail shows its own status: spinning icon (in progress), green tick (completed), red error icon (failed). The button reports progress in real time (e.g. **Syncing... 3/7**). The statuses remain visible for 2.5 seconds after the operation, then disappear.

Footer: destination path

At the bottom of the sidebar the destination download folder path is shown, abbreviated to the last two components (e.g. **Documents / Podcasts**). Clicking this line opens the folder in the system file manager. To change the path, use **Settings → Archive**.

Resizing

The width of the sidebar can be adjusted by dragging the right edge (the cursor becomes a horizontal double arrow). The minimum width is 240 px, the maximum is 640 px, and the default is 360 px. The setting is remembered between sessions.

Connection indicator

The sidebar header shows the connection status: **"Connected"** or **"Offline"**. When the connection is lost, a banner indicates that downloads are paused until it is restored; when the connection returns, the software resumes automatically and checks for new episodes.

3.4 The Episode List

After selecting a feed from the sidebar, the main area is populated with the list of available episodes for that podcast.

Feed header

At the top of the main area the header of the selected feed is visible, with thumbnail, podcast title and episode count. The main batch controls are accessible from this header (see section 3.7).

List columns

Each row in the list represents an episode and contains the following information:

- **Title:** The name of the episode as defined in the RSS feed.
- **Date:** The original publication date of the episode.
- **Duration:** The duration of the episode (when available in the feed).
- **Size:** The file size. Before download, the value is declarative (taken from the feed); after download, it reflects the actual file size.
- **Status:** The visual status indicator for the individual episode. See section 3.5.

Filter bar

Below the feed header there is a filter bar that allows you to narrow the displayed episodes:

- **Text search:** Filters by keywords in the title (AND logic: all entered terms must be present). The filter is automatically cleared when changing feed.

- **Status filter:** Quick buttons to show only episodes in a given status: "All", "Not Downloaded", "Downloaded".
- **Date filter:** "From" and "to" date fields to limit the list to a publication date range.
- **Duration filter:** Limits the list to episodes with a duration between a minimum and maximum (in minutes).
- **Sort:** Opens a panel with five sort options — feed order (default), newest date, oldest date, longest duration, shortest duration.

All filters are automatically cleared when a different feed is selected.

Multiple selection

You can select multiple episodes simultaneously to start their download in bulk:

- **Ctrl+click:** adds or removes the episode from the selection individually.
- **Shift+click:** selects the range between the last selected episode and the clicked one.
- A checkbox appears on hover for unselected episodes and always for selected episodes.

When at least one episode is selected, the "**Download Selected (N)**" button appears in the feed header. The selection is cleared when changing feed and after starting the download.

3.5 Episode Statuses

Each episode in the list is marked with a status indicator. Understanding these statuses is essential for correctly interpreting the state of the archive.

Status	Label in the list	Meaning
Not downloaded	"NEW" tag	The episode is present in the feed but has never been downloaded.
Queued	"queued" entry in the Download Panel	The episode is awaiting its turn in the download queue.
Paused	"paused" entry in the Download Panel	The download has been suspended by the user and can resume from where it stopped.
In progress	Animated progress bar	The download is active. The row shows percentage and speed in real time.
Error	Error summary in the Download Panel	The download failed after all automatic retries.
Downloaded	"ARCHIVED" tag	The file is in the database: downloaded and verified, in this session or a previous one.

*Note on the **"Downloaded"** status:* This status is the result of the Database-First philosophy. When analysing a feed that has previously been processed, most episodes will show the **"ARCHIVED"** tag; the soft-

ware already knows they are present in the archive. Only episodes published after the last download will appear with the **"NEW"** tag.

3.6 Individual Download Controls

On the right of each row in the list, when hovering, episode-specific control buttons appear. The visible buttons vary according to the status:

For all episodes:

- **Copy title** (document icon): Copies the episode title to the system clipboard.
- **Checkbox**: For multiple selection (see section 3.4).

For To Download or Error episodes:

- **Download** (downward arrow): Adds the individual episode to the download queue.

For Downloaded episodes ("ARCHIVED" tag):

- **Re-download** (downward arrow): Adds the episode to the queue again, overwriting the existing file.
- **Forget download** (refresh icon): Clears the episode status, returning it to not-downloaded without deleting the file from disk.
- **Open Folder** (folder icon): Opens the system file manager at the location of the downloaded file.

Interaction with the Detail Panel: A **single click** on the episode row opens the Detail Panel (see section 3.9) with complete metadata and

contextual actions. Ctrl+click and Shift+click are reserved exclusively for multiple selection and do not open the panel.

3.7 Batch Controls

Batch controls operate on the entire download queue, not on individual episodes. They are located in the feed header, above the filter bar.

"Download All": Adds all episodes with the **"NEW"** tag to the queue. Episodes already in the database are excluded automatically. The Download Panel opens automatically on launch.

"Download Selected (N)": Appears when at least one episode is selected. Starts the download exclusively for the selected episodes.

"Stop download": Sends a cancellation signal to all active downloads and empties the queue. Files already completed remain in the database. **.part** files are deleted. Interrupted episodes will appear again with the **"NEW"** tag. For a temporary suspension (without losing progress), use **"Pause"** in the Download Panel instead (see section 3.8).

"Export M3U": Generates a playlist in **.m3u** format with the absolute local paths of all downloaded episodes for that podcast. Opens a native save dialogue. The button is only available when there are downloaded episodes for the current feed.

"Open folder" (folder icon in the header): Opens the file manager in the destination folder for the current feed.

3.8 The Download Panel

The Download Panel is the monitoring and control centre for all downloads in progress. It replaces the previous fixed progress bar at the bottom of the interface.

Opening and closing

The panel opens **automatically** when each batch is started. When closed, the **floating button** (circular icon) is visible in the bottom-right corner of the window: clicking it reopens the panel. Closing the panel does not interrupt downloads in progress.

Panel structure

- **Header:** Shows the completed/total file counter (e.g. `47 / 312`) and the global queue controls: the **"Pause"** button (suspends the entire queue; it becomes **"Resume"** while the queue is paused, and the header shows **"Queue paused"**), the **"Stop download"** button to interrupt all downloads permanently, and the $\times$ button to close the panel.
- **Queue list:** Each download in progress or waiting is represented by a row with: episode title, podcast name, progress percentage, current speed (KB/s or MB/s), individual progress bar. If the server does not declare the file size (no `Content-Length` header), the bar becomes **in-determinate** (a continuous animation) and shows the bytes received. Hovering over the row reveals the **"Pause"** button (or **"Resume"** if paused) and the $\times$ (**"Cancel download"**) button for that individual episode.
- **Non-destructive pause:** Pausing — a single download or the whole queue — keeps the partial `.part` file: on resume, the transfer continues **from where it left off**, without starting over.

- **Errors section:** At the end of the batch, if one or more downloads failed, an expandable summary appears at the bottom of the panel with the list of undownloaded episodes and their error code, together with the **"Retry failed"** button, which re-queues all failed episodes in a single click.
-

3.9 The Episode Detail Panel

The Detail Panel provides an in-depth view of a single episode: meta-data, actions, and — if the episode is already in the archive — technical data from the downloaded file.

Opening the Detail Panel

A **single click** on any row in the episode list opens the panel, which slides in from the right side of the window (below the command bar). The panel closes automatically when a different feed is selected in the sidebar.

Note: Ctrl+click and Shift+click are reserved for multiple selection and do not open the panel.

Panel contents

- **Basic metadata:** Publication date, declared duration, file size as indicated in the feed.
- **Contextual actions:** The available buttons vary according to the episode status: Download, Re-download, Forget download, Open Folder.

- **Archive data** (visible only if the episode has already been downloaded): Date and time of download, actual file size, bitrate, sample rate, filename on disk, SHA-256 checksum.
 - **Source link**: The original URL of the audio file in the RSS feed, with a button to copy it to the clipboard.
 - **Episode notes**: The episode's descriptive text extracted from the feed (show notes), presented in clean text format.
-

3.10 The Archive View

The Archive View is accessible via the **Archive** tab in the sidebar. Unlike the episode list, which shows only the episodes of one feed at a time, the Archive View collects in a single table **all episodes downloaded across the entire library**, regardless of which podcast they belong to.

Features

- **Search**: The search field filters by episode title or podcast name.
- **Podcast filter**: The dropdown menu allows you to limit the display to the episodes of a single podcast.
- **Sorting**: The table can be sorted by download date, publication date, file size and bitrate.
- **Statistics**: The Archive View header shows the total number of downloaded files, the number of distinct podcasts, and the total archive size in gigabytes.
- **Show in folder**: Hovering over a row reveals the button that opens the file manager at the file's location on disk.

The Archive View updates automatically when each download completes.

3.11 The Command Palette (Ctrl+K)

The Command Palette is a quick-access tool that allows you to reach any main function of the software without using the mouse.

Opening the Command Palette

The **Ctrl+K** shortcut (from anywhere in the app, even during a download) opens an overlay with a central search field.

Navigation

- **Typing** in the search field filters actions and feeds in real time.
- **↑↓ arrows** move the selection between results.
- **Enter** executes the selected action.
- **Esc** closes the palette without executing any action.

Contents

- **"Actions" group:** Five fixed commands always available: *Open Settings*, *Sync All*, *Add Feed* (focuses the URL field), *Go to Archive*, *Go to Feeds*.
- **"Feeds" group:** When the search field is empty, shows the first feeds in the library. Typing filters feeds by title. Selecting a feed from the palette loads it directly into the main area.

- **"Episodes (current feed)" group:** As you type, the palette also searches the episode titles of the currently open feed. Selecting an episode filters the list to that title.
-

3.12 The System Tray Icon

When the main window is closed by clicking the X, FeedDownloader Pro does not terminate the process: it minimises to the system notification area (system tray, near the clock). This behaviour is intentional: downloads and the automatic new-episode check continue in the background whilst the window is not visible.

Interacting with the tray icon:

- **Clicking the icon:** Shows or hides the main window.
- **Context menu (right-click):** Contains two entries — **"Show"** (brings the main window back to the foreground) and **"Quit"** (closes the programme and stops all active downloads).

Practical note: To run a large download without keeping the window open, start the batch, close the window, and leave the computer running. The archive will be available once the process completes.

Go to Chapter 4 for a complete workflow from the first analysis to download.

Chapter 4: The First Archive — Step-by-Step Guide

4.1 Introduction to the Workflow

This chapter describes a complete workflow, from a podcast URL to an organised archive on disk. The reference scenario is the most common one: downloading the entire catalogue of a podcast for the first time.

It is recommended to read the chapter from beginning to end at least once. Once familiar with the steps, starting a new archive takes less than a minute.

4.2 Phase 1: Finding the RSS URL

The starting point is the RSS feed URL of the podcast to archive. An RSS feed is a text file in XML format that podcast services publish to distribute the list of available episodes. Every podcast has an RSS feed.

How to find the RSS URL:

- **On the podcast's website:** Look for an orange icon with radio waves, or the text "RSS", "Feed", "Subscribe", or "Podcast Feed". Clicking on the element generally opens the XML file in the browser: the URL displayed in the address bar is the one to use.

- **From a podcast app:** Applications such as Pocket Casts, Apple Podcasts, and similar often show the RSS link in the podcast information. On some apps the link is accessible via the "Share" function.
- **From hosting services:** If the podcast is hosted on Spreaker, Podbean, Buzzsprout, or equivalent platforms, the feed URL is usually available in the publisher panel or in the podcast's public information.
- **From a search engine:** Search for `[Podcast Name] RSS feed`. The first result often leads directly to the correct URL.

How to recognise a valid RSS URL: It generally ends with `.xml` or `.rss`, or contains words such as `feed`, `rss`, or `podcast` in the path. Examples:

`https://www.example.com/feed.xml` ,
`https://feeds.spreaker.com/podcast/12345` ,
`https://anchor.fm/s/abc123/podcast/rss` .

4.3 Phase 2: Preparing the Destination Folder

Before analysing the feed, it is advisable to define the destination folder. It is recommended to create an organised structure from the outset.

Recommended structure: `D:\Podcast Archive\ |— My Podcast\ |— Technology Podcast\ |— Radio Talk Show\`

Create the specific folder for the podcast to be archived (e.g. `D:\Podcast Archive\My Podcast\`). FeedDownloader Pro will save all files from that podcast into that folder, with names defined by the rename template (see Chapter 8).

To set the destination folder in FeedDownloader Pro:

1. Open **Settings** → **Archive** and click the **folder** icon next to the destination path field.
2. Navigate to the created folder and select it.
3. The configured path is always visible in the footer of the left-hand sidebar; clicking it opens the folder directly in the file manager.

Note: For paths on NAS or network drives, consult Chapter 7 before proceeding. The configuration for network paths has some specificities described in that chapter.

4.4 Phase 3: Analysing the Feed

With the URL ready and the destination folder set:

1. Paste the RSS URL into the **URL field** at the top of the interface.
2. Click "**Analyse**" (or press **Enter**).
3. The list in the centre is populated with the episodes. For a podcast with 200–300 episodes, the operation typically takes 2–5 seconds. For very large archives (1,000+ episodes), up to 15–20 seconds may be required, as the feed's XML file can reach considerable sizes.

In the event of an analysis error:

- Check that the URL is correct (no leading or trailing spaces, no missing characters).
- Open the URL in the browser: if the browser returns an error or a blank page, the feed may be temporarily unavailable or the URL may have changed.

- Some feeds require specific HTTP headers. In this case the software displays an error message with the HTTP code received (for example `403 Forbidden`).
-

4.5 Phase 4: Reading the Analysis Results

After the analysis, the list shows all episodes of the podcast.

Items to verify:

- **Total number of episodes:** Visible in the list header or in the counter at the bottom. A podcast active for several years may have 300–500 episodes or more.
 - **Episodes in "Downloaded" status:** If the podcast has been analysed previously, most episodes will appear in this status. The database already records these files as present in the archive.
 - **Episodes with missing data:** It is possible that some episodes do not show duration or size. This indicates that the podcast producer did not include this information in the RSS file. The download proceeds correctly in any case.
-

4.6 Phase 5: Starting the Download

Two download modes are available.

Mode A — Full download: Click "**Download All**". The software adds all episodes with the "**NEW**" tag to the queue and starts the downloads in

parallel. The number of simultaneous downloads depends on the **"Parallel Downloads"** setting (see Chapter 10; the default value is 3).

Mode B – Selective download: To download only certain episodes:

1. Select the episodes by holding **Ctrl** and clicking on each one.
 2. To select a range, click on the first episode, hold **Shift**, and click on the last.
 3. Click the **"Download Selected (N)"** button that appears in the feed header when at least one episode is selected.
-

4.7 Phase 6: Monitoring Progress

During the download:

- **Download Panel:** Opens automatically on the right side of the window when the batch starts. It shows each queued episode with percentage, current speed, and estimated time to completion. For an archive of 200 episodes at an average of 64 kbps, the total data volume is approximately 2–3 GB.
- **Status in the list:** Each row updates in real time. Episodes in progress show an individual progress bar with the completed percentage.
- **Background execution:** It is not necessary to keep the window open. It can be closed (the programme continues to operate in the system tray) and reopened when the process has completed.

The software automatically handles retries in the event of a network error, stall detection in the case of slow servers, and integrity verification upon completion of each file. If the computer enters sleep mode, down-

loads are interrupted and automatically resumed when the session is restored.

4.8 Phase 7: Verifying the Completed Archive

When the Download Panel shows the batch as completed and all episodes appear in green status, the archive is ready.

Recommended operations upon completion:

1. **Check for errors:** If some episodes show "**Error**" status (red), click them to open the Episode Detail Panel and read the error code. Alternatively, consult the error summary section at the bottom of the Download Panel. The most common cause is **404 Not Found**, which indicates the file was removed from the podcast server before the download.
 2. **Export a CSV summary:** Go to **Settings** → **Archive** → **Export CSV**. The generated file lists all downloaded episodes with SHA-256 hashes, sizes, and metadata (see Chapter 9).
 3. **Verify files on disk:** Open the destination folder in the file manager. Audio files are organised according to the configured rename template (see Chapter 8). The presence of **.part** files indicates interrupted or paused downloads: re-queueing the same episodes resumes the transfer from where it stopped. Orphaned **.part** files can be removed via **Settings** → **Advanced** → **Clean temporary files**.
-

4.9 Updating the Archive in Future

The Database-First system simplifies archive updates. The procedure depends on whether the feed is already in the library or not.

Feed already in the sidebar:

1. Click the feed in the sidebar to select it.
2. Hover over the entry and click the sync icon, or use the **"Sync All"** button to refresh the whole library in parallel.
3. New episodes appear with the **"NEW"** tag (and the feed shows the **"TO DOWNLOAD"** badge in the sidebar); those already present remain **"ARCHIVED"**.
4. Click **"Download All"** to download only the new episodes.

Feed not yet in the library:

Paste the RSS URL into the URL field at the top of the interface and click **"Analyse"**: the feed is added to the library and the list is populated with the current status.

The system never downloads the same episode twice. The automatic new-episode check is enabled by default every 6 hours (see Chapter 10 and section 5.9).

Go to Chapter 5 to learn more about feed management and OPML features.

Chapter 5: Feed Management

5.1 What is an RSS Feed

An RSS feed is an XML document published by a podcast to allow applications to automatically read the list of available episodes. When a publisher releases a new episode, they update this document by adding a new entry. Podcast applications periodically read these documents to identify the most recent content.

For FeedDownloader Pro, the RSS feed is the **primary data source**: it contains the episode list, audio file URLs, metadata (title, date, duration, description, cover art), and general podcast information (name, author, category).

Knowledge of the internal structure of an RSS feed is not necessary to use the software, but it facilitates the interpretation of data displayed in the episode list and the understanding of why some information may be missing or incomplete.

5.2 Valid Feeds and Problematic Feeds

Not all RSS feeds comply with the same level of adherence to the standards.

Well-formed feed: Follows the RSS 2.0 or Atom standard, includes all required fields (title, link, publication date, audio URL with MIME type), and optionally the iTunes/Podcast Index tags for duration, cover art, and seasons. FeedDownloader Pro reads these feeds without issue.

Partially incomplete feed: Some optional fields are missing (duration, file size, episode cover art). The software downloads the audio files regardless, but some columns in the list will remain empty.

Feed with unreachable audio URLs: The feed is readable, but the audio file URLs point to resources that no longer exist (404 error). This situation is common with abandoned podcasts or those migrated to other servers. FeedDownloader Pro marks these episodes with **"Error"** status after the download attempt.

Authentication-protected feeds: Some private or paid podcasts require HTTP Basic credentials to access the feed. The software supports these feeds: credentials are included directly in the URL in the format `https://username:password@www.example.com/feed.xml` .

5.3 Analysing a Feed: Detail

When **"Analyse"** is clicked, FeedDownloader Pro performs the following operations in sequence:

1. **URL validation:** Verifies that the URL is syntactically correct and passes the 5 anti-SSRF checks (see Chapter 10 for details).
2. **HTTP request:** Contacts the feed server with a standard user-agent. The timeout for this operation is 30 seconds.

3. **XML parsing:** Reads and analyses the RSS or Atom document. The software handles feeds with minor deviations from the standards (undeclared encoding, missing tags, unconventional namespaces).
 4. **Deduplication:** For each episode in the feed, the database is queried to verify whether the episode has already been downloaded. The audio URL is used as the unique identification key.
 5. **List population:** All episodes are displayed with their current status.
 6. **Adding to the library:** The feed is permanently inserted into the sidebar if it is not already present. Feeds already in the library are updated with the latest episode count.
-

5.4 The Feed Library

FeedDownloader Pro maintains a **permanent feed library**. Each analysed feed is saved in the sidebar and remains available between sessions, with no need to re-enter the URL at every launch.

Display

Each item in the library shows: the podcast cover art (thumbnail), the title, the date of the last synchronisation with the server, and the **"TO DOWNLOAD"** badge (e.g. **3 TO DOWNLOAD**), indicating how many new episodes have been detected via their unique identifier (GUID). The badge disappears as soon as all the new episodes have been downloaded.

Clicking on a feed in the sidebar immediately updates the episode list in the main area.

Removing a feed from the library

To remove a feed, hover over the entry in the sidebar: the bin button appears on the right side of the row. Clicking it opens a confirmation dialog. Removal deletes the feed from the library but **does not delete the audio files already downloaded** nor the related data in the database; the episodes remain visible in the Archive View.

Search and sorting

- **Feed search:** The search field at the top of the sidebar filters feeds by name in real time. Useful with large libraries.
- **A-Z sorting:** The sort button arranges feeds alphabetically by title. Clicking it again restores the original order.

Privacy note: The feed library is saved exclusively in the local database. No data is transmitted to external servers.

5.5 Importing Feeds from OPML

OPML (Outline Processor Markup Language) is the standard format for exporting and importing podcast lists between different applications. If you have a podcast library in an app such as Pocket Casts, Overcast, AntennaPod, or any other client, you can export it to OPML and import it directly into FeedDownloader Pro.

How to import an OPML file:

1. Go to **Settings** → **Archive**, "Data and Portability" section.

2. Click **Import Feeds (OPML)** and select the `.opml` file exported from the podcast application.
3. FeedDownloader Pro analyses the file and adds the identified feeds to the library.

Note: Some podcast applications use proprietary variants of the OPML format. FeedDownloader Pro supports the most widely used versions. If a file is not imported correctly, open it with a text editor and verify the presence of `<outline type="rss" xmlUrl="...">` tags for each podcast.

5.6 Exporting the Library to OPML

The feed library can be exported in OPML format to:

- Create a backup of the podcast list.
- Share it with other users or with another installation of the software.
- Import it into a podcast application to follow the same feeds.

How to export:

1. Go to **Settings** → **Archive**, "Data and Portability" section.
 2. Click **Export Feeds (OPML)** and choose a name and location for the file.
 3. The generated file is compatible with any application that supports the OPML standard.
-

5.7 Large Feeds

Some historical podcasts or radio production archives can have feeds with thousands of episodes and RSS files of considerable size. In these cases:

- **RFC 5005 pagination:** Many platforms publish only the most recent episodes in the feed and split the historical archive into multiple linked pages (RFC 5005 standard, `rel="next"` links). FeedDownloader Pro follows these links automatically and reassembles the entire catalogue into a single list, with no user intervention. Every pagination link is subject to the same security checks as manually entered URLs.
 - **The initial analysis takes longer:** A feed with 2,000 episodes (or split across many pages) may require 15–30 seconds for download and parsing. This behaviour is expected.
 - **List virtualisation:** With thousands of entries, the list loads only the rows visible on screen to keep the interface responsive.
 - **Estimating required space:** With 2,000 episodes at approximately 50 MB each, the total volume is approximately 100 GB. Verify that sufficient space is available before proceeding.
-

5.8 Managing Multiple Feeds

FeedDownloader Pro natively manages a library of multiple feeds. There is no limit to the number of podcasts that can be added: all are kept in the sidebar and remain accessible between sessions.

Navigating between feeds

Clicking on a feed in the sidebar immediately updates the episode list in the main area. The software remembers which feed was selected at the last closure.

Synchronising feeds

- **Individual synchronisation:** Hover the mouse over a feed entry in the sidebar to display the sync icon. Clicking it makes the software re-read that feed from the server and update the list with any new episodes.
- **Sync All:** The "**Sync All**" button refreshes all feeds in parallel in a single operation. During the process, each thumbnail in the sidebar shows its own status in real time. On completion, any new episodes are highlighted with the "**TO DOWNLOAD**" badge.

For scheduled automatic refresh without manual intervention, see section 5.9.

5.9 Automatic Feed Refresh

FeedDownloader Pro synchronises all feeds automatically in the background, without requiring any action from the user. The check runs:

- **at startup** (a few seconds after the application opens);
- **at regular intervals** (the "Automatic Feed Refresh" setting);
- **when the connection returns** after a period offline.

Configuration

The setting is found in **Settings** → **General** → **Automatic Feed Refresh**. Four options are available:

Option	Behaviour
Disabled	No periodic synchronisation (the startup check remains).
6 hours (default)	The software synchronises all feeds every 6 hours.
12 hours	The software synchronises all feeds every 12 hours.
24 hours	The software synchronises all feeds once every 24 hours.

The setting change is immediate and does not require restarting the software. The timer starts from application startup.

Behaviour

Automatic refresh **does not start downloads**: it only checks whether new episodes have been published. If new episodes are found on one or more feeds, the system sends an **operating system notification** with a summary of the content found (in Italian or English, following the interface language). The notification is **clickable**: it brings the application to the foreground and, if the new episodes belong to a single podcast, opens that feed directly.

To download the reported new episodes, use the normal batch controls.

Go to Chapter 6 for a detailed description of the download engine.

Chapter 6: The Download Engine

6.1 Engine Architecture

The download engine of FeedDownloader Pro is an asynchronous multi-threaded system. Unlike a sequential downloader, the software manages multiple downloads simultaneously through a central queue system.

Main components:

- **The queue:** An ordered list of all pending downloads. Every episode added to the batch enters this queue and waits to be assigned to an available thread.
 - **Worker threads:** The processes that physically execute the downloads. The number of active threads is configurable. Each thread handles one download at a time, independently of the others.
 - **The database manager:** The component that updates the SQLite database in real time with the status of each download (started, completed, failed, completion percentage).
 - **The integrity monitor:** The process that, upon completion of each download, calculates and records the SHA-256 hash of the downloaded file.
-

6.2 Parallel Downloads: Configuration

The number of simultaneous downloads is one of the most relevant parameters to configure. An insufficient value slows the process; an excessive value can saturate the connection, overload the source server, or generate network errors.

The default value is 3 threads. For most users with a home connection, this value offers a good balance between speed and stability.

Configuration guidelines:

Scenario	Recommended threads
Slow connection or server with throttling	1
Standard home connection	3 (default)
Fast fibre connection	5
NAS with slow network connection	1

How to change the number of simultaneous downloads: Go to **Settings** → **Download** → **Parallel Downloads** and select one of the three available presets: **1**, **3**, or **5**. The change is applied immediately to the current queue.

Note on servers with connection limits: Some podcast hosting servers apply limits to the number of simultaneous connections per single IP address. If frequent **429 Too Many Requests** or **503 Service Unavailable** errors occur, reduce the number of threads to 1 or 2. The retry mechanism automatically handles failures, but reducing the load prevents the problem at its root.

6.3 Error Handling and Retry System

In a batch download of hundreds of files, network errors are to be expected. FeedDownloader Pro uses an **exponential backoff retry** strategy: when a download fails, the system waits an increasing interval before retrying, rather than immediately re-queuing the episode.

Retry cycle:

Attempt	Wait before retry
1st failure	1 second
2nd failure	2 seconds
3rd failure (last)	The episode is marked as a definitive "Error"

If a server is temporarily overloaded, the system gives the server time to recover before retrying. If the server replies **429 Too Many Requests** with a **Retry-After** header, the wait requested by the server is honoured (up to a maximum of 60 seconds). Because the partial **.part** file is kept between attempts, each retry **resumes from where it stopped** instead of starting over.

Definitive errors (not subject to retry):

- **404 Not Found**: The file does not exist on the server. Further attempts are not useful.
- **Non-audio content**: The server replied with a web page (HTML) instead of the audio file — typical of expired links redirecting to a

courtesy page. The download is rejected with the message "The server sent a web page, not audio".

- **File over the size cap:** The file exceeds the "**Maximum File Size**" value configured in the Settings (see Chapter 10).
- **Disk full or access denied** on the destination folder.
- SSRF validation errors: The URL did not pass the internal security checks.

Retrying failed episodes: At the end of the batch, the "**Retry failed**" button in the errors section of the Download Panel re-queues all failed episodes in a single click.

6.4 Stall Detection

A stalled download is a scenario in which the TCP connection is technically active and packets continue to arrive, but the data flow has stopped. The operating system does not report errors as the connection is still open; the file continues to show as "downloading" without progressing.

This condition occurs frequently with:

- Servers under load that apply throttling after sending the first bytes.
- Intermediate network routing issues.
- Large audio files served from CDN with bandwidth limitations.

Detection: Each active download is monitored by a watchdog. If no new bytes arrive for **60 consecutive seconds**, the download is considered stalled and:

1. The connection is terminated.
2. The partial `.part` file is **kept**.
3. The episode re-enters the normal retry cycle and, thanks to the partial file, the new attempt **resumes from where it stopped**.

The process is transparent to the user. If the stall was caused by a transient condition, the download resumes normally. If the problem persists beyond the maximum attempts, the episode is marked as "**Error**".

6.5 `.part` Files: Anti-Corruption and Resume

Every audio file is downloaded with the temporary `.part` extension during transfer. The file is renamed with the final extension (`.mp3` , `.m4a` , `.ogg` , etc.) **only** after:

1. The transfer is 100% complete.
2. The file size matches the size declared in the HTTP header (`Content-Length`), if available.
3. The SHA-256 hash has been calculated and recorded in the database.

This mechanism guarantees that partial or corrupted audio files with a final extension are never present in the destination folder.

Resuming transfers: In the event of a pause, transient error or interruption, the `.part` file is kept together with a small `.part.meta` file recording the server's "validator" (ETag or Last-Modified). On the next attempt, the software requests only the missing bytes (HTTP `Range` request with `If-Range`); if the remote file has changed in the meantime, the server signals it and the download restarts from scratch, avoiding splicing fragments of different files together.

Cleaning up leftovers: `.part` files orphaned by past sessions can be removed via **Settings** → **Advanced** → **Maintenance** → **Clean temporary files** (the function is only available while no downloads are running).

Location of `.part` files: In the same destination folder as completed files. These files must not be opened with an audio player: being partial, they would cause read errors.

6.6 Pausing, Resuming and Stopping

Pausing (non-destructive): From the Download Panel you can suspend a **single download** (the "Pause" button on its row) or **the entire queue** (the "Pause" button in the header; the panel shows "Queue paused"). Pausing keeps the `.part` files: pressing "Resume" continues the transfer exactly from where it left off. A paused download keeps its place in the queue and in the batch.

Stopping the Batch (destructive): The "Stop download" button in the Download Panel stops all active downloads in an orderly fashion, clears the queue, and deletes partial `.part` files. Files already completed remain in the database. Interrupted episodes will appear again with the "NEW" tag.

Closing the window during a download: Closing the main window with the X keeps the programme running in the system tray, and downloads continue in the background. The tray menu's "Quit" entry, instead, closes the programme for good, stopping active downloads; the `.part` files remain on disk, so re-queueing the same episodes in the next session resumes the transfer from where it stopped.

6.7 Download Speed

The overall batch speed is the **aggregate sum** of all active downloads. With 3 active threads each downloading at 2 MB/s, the total speed displayed is approximately 6 MB/s.

Factors affecting speed:

- **Connection bandwidth:** The maximum available limit.
- **Source server speed:** Many podcast hosting servers apply bandwidth limitations to contain costs. The speed of a single thread rarely exceeds 2–5 MB/s on these servers.
- **Number of threads:** A greater number of threads compensates for the slowness of individual servers by downloading from multiple simultaneous connections.
- **File size:** Average-sized files (20–80 MB, corresponding to 30–60 minute episodes) offer optimal efficiency, with a reduced relative connection overhead.

Go to Chapter 7 for the configuration of NAS and network paths.

Chapter 7: NAS, Network Drives, and SMB Paths

7.1 Why Network Drives Require a Specific Approach

Most desktop download applications handle local paths (`C:\` , `D:\`) correctly and exhibit unpredictable behaviour when the destination is a NAS, a shared Windows server, or an SMB unit. The reason is technical: network drives are inherently less reliable than local drives. The NAS may be switched off, the local network may experience latency spikes, SMB credentials may expire. Any operation on a network path that is not responding can block the application's main thread for tens of seconds, rendering the interface unresponsive.

FeedDownloader Pro handles these scenarios correctly. For users who archive to NAS, this chapter is essential.

7.2 How Network Path Validation Works

Every time a destination path is set that begins with `\\` (UNC path, typical of SMB) or corresponds to a mapped network drive (e.g. `Z:\`), FeedDownloader Pro automatically activates the **network path validation module**.

This module performs three operations on a **separate thread**, without ever involving the interface thread:

1. **Reachability test:** Attempts to access the root of the network path. If the NAS is not switched on or the network is unavailable, this operation fails.
2. **Read access test:** Verifies that the destination folder exists and is readable.
3. **Write access test:** Creates and then deletes a temporary file (`_fdp_write_test_[timestamp].tmp`) in the destination folder to verify write permissions.

The entire sequence has an **8-second timeout**. If no response is received within this interval, the software considers the path unavailable and displays a warning, without blocking the interface.

Rationale for the timeout: Most consumer NAS devices (Synology, QNAP, WD MyCloud) take 3–6 seconds to wake from sleep mode. 8 seconds is a sufficient interval to await this recovery, whilst remaining short enough not to constitute a perceptible wait for the user.

7.3 Configuring a NAS Path

Method 1 — Direct UNC path: Enter the path in the format `\\ServerName\ShareName\Folder` :

```
\\MYNAS\Podcast\Archive
\\192.168.1.100\media\podcast
\\NAS-SYNOLOGY\video\audio_archive
```

The path can be entered directly in the destination text field, or via the folder selection window, which on Windows supports navigation of network paths.

Method 2 — Mapped network drive: If the NAS is already mapped as a network drive in Windows (e.g. **Z:** → **\\MYNAS\Podcast**), it is possible to select **Z:\Archive** as the destination folder. FeedDownloader Pro automatically recognises that this is a network path and activates validation.

Method 3 — Linux (mount point): On Linux, SMB network paths are presented as normal folders in the filesystem after mounting (e.g. **/mnt/nas/podcast**). These paths can be used directly as the destination folder.

7.4 SMB Credentials and Authentication

NAS access credentials must be configured at the operating system level, not within FeedDownloader Pro.

On Windows:

1. Open **File Explorer** and navigate to the NAS path (**\\MYNAS**).
2. Enter the credentials when prompted and tick "**Remember my credentials**".
3. The credentials are saved in the **Windows Credential Manager** (**Control Panel** → **Credential Manager** → **Windows Credentials**).
4. FeedDownloader Pro, like any other application, will access the NAS without requiring further credentials.

On Linux: Mount the share with credentials in the `fstab` file or via a graphical tool such as GNOME Files. Alternatively, use `smbclient` or `mount -t cifs` from the terminal.

7.5 Diagnosing Problems with Network Paths

In the event of a "Network path not reachable" warning, check the following points in the order shown.

1. Is the NAS switched on and started up? Check the device indicator lights. Many consumer NAS devices enter sleep mode after a period of inactivity. Before starting the download, open the NAS administration panel from the browser to verify its availability.

2. Is the NAS reachable from the network? From the Command Prompt (Windows) or Terminal (Linux): `ping 192.168.1.100` Replace with the NAS IP address. If the command receives a reply, basic network connectivity is working.

3. Is the SMB share accessible? Attempt to open the path `\\192.168.1.100\ShareName` directly from Windows File Explorer. If the operation fails, the problem lies in the NAS SMB configuration, not in FeedDownloader Pro.

4. Are write permissions correct? Manually create a file in the destination folder via the file manager. If the operation is not permitted, the user used to access the NAS does not have write permissions on that share. Configure the permissions from the NAS administration panel.

5. Is the firewall blocking SMB connections? The SMB protocol uses port 445 (and in some cases port 139). Verify that the system or third-

party firewall is not blocking these ports for connections on the local network.

7.6 Optimal Performance on NAS

Downloads to NAS present an additional complexity compared to those to a local drive: files are written across the network and speed depends on both the LAN bandwidth and the NAS write capacity.

Operational guidance:

- **Use a wired (Ethernet) connection:** Wi-Fi introduces latency and instability in network write operations. For large archives, a wired Gigabit Ethernet connection offers significantly better performance.
 - **Reduce parallel downloads:** Simultaneously writing many files to a NAS can saturate its I/O. A "Parallel Downloads" value of 1–3 often yields better results than using the maximum available number.
 - **Avoid overlapping with NAS backups:** If the NAS runs automatic backups, avoid starting batch downloads during the same time windows, as competition for the disk's I/O slows down both operations.
 - **Use a local cache:** For very large archives, it is possible to download first to a fast local drive and move the files to the NAS once the download is complete.
-

Go to Chapter 8 for the configuration of the rename template and meta-data features.

Chapter 8: File Organisation, Templates, and Metadata

8.1 The Problem of Non-Meaningful Names

When an audio file is published on a podcast server, its original name is often difficult to read: `ep_2024_03_15_FINAL_v2_mixdown.mp3`, `podcast-episode-187-compressed.m4a`, or even just `abc123def456.mp3` are common examples. These names make sense for the producer's systems, but make an archive difficult to browse.

FeedDownloader Pro resolves this problem through the **rename template system**: a mechanism that allows you to define a custom name format for all downloaded files, using information extracted directly from the RSS feed.

8.2 How the Template Works

A rename template is a text string that can contain fixed text and **tokens** – variables enclosed in single curly braces (`{ }`). Upon completion of each download, the software replaces each token with the corresponding value for the episode.

Example:

Configured template: `{date} - {podcast} - {title}`

Result: 2024-03-15 - Mario's Podcast - Episode 187: Artificial Intelligence Explained.mp3

The file extension (.mp3 , .m4a , etc.) is added automatically based on the format of the original file: it is not part of the template.

8.3 Available Tokens

Token	Description	Example
{title}	Episode title from the RSS feed	Episode 187: AI Explained
{pod-cast}	Podcast name (RSS channel title)	Mario's Podcast
{date}	Publication date in YYYY-MM-DD format	2024-03-15
{year}	Publication year	2024
{month}	Publication month (2 digits)	03
{day}	Publication day (2 digits)	15

Note: If text is entered in the template within curly braces that does not correspond to any of the listed tokens (for example {episode}), the text is left unchanged in the resulting file name.

8.4 Recommended Templates

Default template: `{title}` The default template uses the episode title alone. It is suitable for catalogues with descriptive titles.

For general use (recommended): `{date} - {title}` Result: `2024-03-15 - Episode 187: AI Explained.mp3`

This format is recommended because alphabetical sorting of the files coincides with chronological ordering.

For multi-podcast archives (shared folder): `{podcast} - {date} - {title}` Result: `Mario's Podcast - 2024-03-15 - Episode 187.mp3`

Useful when all podcasts are saved in the same destination folder.

For organisation into subfolders by year and month: `{year}/{month}/{date} - {title}` Creates an automatic subfolder structure (see section 8.7).

8.5 Automatic Name Normalisation

Some characters are not permitted in file names on the main operating systems: `/`, `\`, `:`, `*`, `?`, `"`, `<`, `>`, `|` on Windows.

FeedDownloader Pro automatically applies **normalisation** to the name produced by the template:

- Non-permitted characters are replaced with a hyphen (-) or removed.
- Double spaces are reduced to a single space.

- Leading and trailing hyphens or spaces are removed.
- The name is truncated to 240 characters if it exceeds the filesystem limit.

Note on long titles: Some podcasts use very descriptive titles (over 150 characters). Using the `{title}` token in the template may produce very long file names. In these cases, pairing `{date}` as the primary chronological element can limit the overall length of the name.

8.6 Configuring the Template

The rename template is configured in **Settings** → **Metadata**, in the "**File Naming Template**" field.

The text field accepts any combination of text and tokens. Beneath the field, a real-time preview is available showing the result of the template applied to a sample episode, to verify the format before saving.

The default template is `{title}`.

8.7 Organisation into Subfolders

In the template it is possible to use the `/` character to create an automatic **subfolder** structure within the destination folder.

Example — organisation by year and month: `{year}/{month}/{date} - {title}`

With a destination folder of `D:\Podcast Archive\Mario's Podcast\`, the result will be: `D:\Podcast Archive\Mario's Podcast\ | — 2024\ | | — 01\ | | — 2024-01-08 - First Episode of the Year.mp3 | | — 2024-01-22 - Second Episode.mp3 | — 03\ | — 2024-03-15 - Episode 187.mp3 — 2023\ — 12\ — 2023-12-20 - Last Episode of 2023.mp3`

Subfolders are created automatically if they do not exist.

Caution: The `\` (backslash) character is not supported as a path separator in the template. Always use `/` (forward slash), which the software correctly translates for the operating system in use.

8.8 Sidecar JSON Files

The **"Sidecar .json File"** toggle is available in the **Settings** → **Metadata** tab.

When enabled, for each downloaded audio file a `.json` file is created with the same name in the same folder. The file contains the episode metadata in structured format:

```
{
  "title": "Episode 187: AI Explained",
  "podcast": "Mario's Podcast",
  "date": "2024-03-15",
  "sourceUrl": "https://media.example.com/ep187.mp3"
}
```

Use cases:

- Integration with automation scripts or systems that read metadata directly from the filesystem without querying the database.
- Preserving metadata independently of the database, useful in the event of archive migration or reconstruction.

This option is disabled by default.

8.9 ID3 Tagging

The "ID3 Tagging" toggle is available in the **Settings** → **Metadata** tab.

When enabled, upon completion of each download the software writes metadata directly into the `.mp3` file, in the standard ID3 tags:

- **Title:** The episode title
- **Artist:** The podcast name
- **Year:** The publication year
- **Cover art:** The podcast image (if available in the RSS feed)

ID3 tags are recognised by major audio players (Windows Media Player, VLC, iTunes, Foobar2000) and allow episode information to be displayed independently of the file name.

Note: ID3 tagging applies exclusively to `.mp3` files. Files in other formats (`.m4a`, `.ogg`, `.opus`) are not modified, even with this option active.

This option is disabled by default.

Go to [Chapter 9](#) for integrity verification and archive management.

Chapter 9: Integrity, Statistics, and Archiving

9.1 Why Verify File Integrity

The completion of a download does not guarantee that the received file is intact. A network packet lost during transfer, a disk write error, or an interruption in the last second can produce a file that is formally "present" but corrupted. Without an explicit verification, an apparently complete archive can contain unplayable audio files whose corruption is only detected during playback.

FeedDownloader Pro addresses this problem with two complementary mechanisms: **size verification** (during download) and **SHA-256 verification** (upon completion).

9.2 SHA-256 Verification

SHA-256 (Secure Hash Algorithm 256-bit) is a cryptographic function that produces a 64-character hexadecimal fingerprint for any file. Two identical files always produce the same hash; a difference of even a single bit produces a completely different hash.

For each downloaded file, FeedDownloader Pro:

1. Calculates the SHA-256 hash of the file at the end of the download.

2. Saves the hash in the database, together with the file path and the date of calculation.

Practical uses:

- The **Health Check** (see section 9.4) recalculates the hash of every present file and compares it with the recorded one: any modification, corruption or replacement after the download is detected.
- The **"Repair archive (checksum search)"** function uses the same hash to find files that have been renamed by hand (see section 9.4).
- In professional contexts, the SHA-256 hash constitutes a verifiable reference of content integrity at the time of download.

9.3 Extracted Audio Metadata

Upon completion of each download, FeedDownloader Pro automatically extracts the **technical metadata** from the audio file. This information is read directly from the file (not from the RSS feed) and recorded in the database.

Extracted metadata:

Field	Description	Example
Bitrate	Audio quality in kilobits per second	128 kbps , 320 kbps
Sample rate	Sampling frequency	44100 Hz , 48000 Hz

Field	Description	Example
Size on disk	Actual size of the downloaded file	67.4 MB

These values are recorded in the database and are included in the CSV export (see section 9.6).

9.4 Health Check: Archive Integrity Verification

Over time, an archive may undergo external modifications outside the software: files moved, renamed, deleted or corrupted directly on the filesystem. The **Health Check** verifies the state of the archive against what is recorded in the database.

How to run the Health Check: Go to **Settings** → **Archive** → **Archive Health Check** and click **"Run Check"**.

The process performs two checks on each file recorded in the database:

1. **Presence:** verifies that the file still exists at the recorded path.
2. **Real integrity:** for present files, it recalculates the SHA-256 hash and compares it with the one recorded at download time. A modified or damaged file is reported as **corrupted** ("checksum mismatch"), with the suggestion to re-download the affected episodes.

Upon completion, a summary is shown with four indicators:

Indicator	Meaning
In DB	Total number of episodes in the database

Indicator	Meaning
On Disk	Files that exist at the recorded path
Missing	Files not found at the recorded path
Disk Usage	Total disk space of the present files

If missing files are found, the software lists the first 5 with the podcast name and file name, and offers two actions:

- **"Repair archive (checksum search)":** Searches the podcast folders for files that have been **renamed by hand**: each candidate file is identified via its recorded SHA-256 hash and, on a match, is re-linked to the database under its new name — without re-downloading anything. A summary is then shown: "*N files re-linked, M not found*".
- **"Mark as not downloaded":** Removes the genuinely missing files from the registry; they return to the episode list with the **"NEW"** tag and can be re-downloaded with the normal controls ("**Download**" or "**Download All**").

9.5 Archive Statistics

The statistics section is accessible from **Settings** → **Archive** and provides a concise overview of the data recorded in the database:

- **Downloaded files:** Total number of episodes present in the database.
- **Podcasts:** Number of distinct feeds represented in the archive.
- **Time range:** Date of the first and last downloaded episode.

Statistics are automatically updated each time the Settings panel is opened.

9.6 CSV Export

The CSV export generates a file with the data of each episode present in the database. It is useful for integrating FeedDownloader Pro with other tools (spreadsheets, content management systems, automation scripts).

How to export: Go to **Settings** → **Archive** → **Export CSV** and choose the path in which to save the file.

Export columns:

Column	Content
Podcast	Podcast name
Episode Title	Episode title
Publish Date	Publication date
Downloaded At	Date and time of download
File Size (bytes)	File size in bytes
Bitrate (kbps)	Audio bitrate in kilobits per second
Sample Rate (Hz)	Sampling frequency in hertz
SHA-256 Checksum	SHA-256 hash of the file

Column	Content
Validation Status	Result of the last integrity check
GUID	Unique identifier of the episode in the RSS feed

File format: CSV with comma separator (,), UTF-8 encoding with BOM (for compatibility with Microsoft Excel). Fields containing commas are enclosed in quotation marks.

9.7 Archive Migration

To move the archive to a new drive or a new folder, use the built-in migration function, which keeps the database synchronised with the new file location.

Procedure:

1. Go to **Settings** → **Archive** → **Migrate Archive**.
2. Select the **new destination folder** via the selection window.
3. The software physically moves all audio files to the new folder and updates the paths in the database.
4. Upon completion, a summary is shown: number of files moved and any errors.

Caution: The migration moves files from the current folder to the new one. Files are removed from the original location. Verify that the destination drive has sufficient space before starting the operation.

Moving to a new computer: Copy both the audio files folder and the `feeddownloader.sqlite` file (from the user data folder described in Chapter 2). On the new computer, install FeedDownloader Pro, copy the database to the user data folder, and use the migration function if the archive path has changed.

Go to Chapter 10 for the advanced software settings.

Chapter 10: Advanced Settings

10.1 Overview of the Settings Panel

The settings panel is accessible at any time via the gear icon (⚙️) in the upper corner of the interface. Settings are organised into five thematic tabs: **General**, **Download**, **Metadata**, **Archive**, and **Advanced**. All changes are saved automatically: it is not necessary to confirm with a dedicated button.

10.2 Download

This section contains the main controls for the download engine. Internal technical parameters (connection timeout, number of retries, stall detection) are fixed in the engine and do not require manual configuration.

Parallel Downloads

The number of simultaneous downloads. Selectable from three presets: **1**, **3**, and **5**. For guidelines on choosing the value, see Chapter 6.

Default value: 3

Download Speed Limit

Allows you to limit the aggregate bandwidth used by all active downloads, to avoid interference with other network activities.

Available values: `0` = unlimited (default); any positive value in KB/s.
Example: `500` limits total consumption to approximately 4 Mbps.

Maximum File Size

Rejects downloads larger than the configured cap, in **MB**. It protects against anomalous enclosures (huge files declared by mistake, or servers sending content other than the expected audio). A file over the cap is marked as a definitive error, with no retry.

Available values: `0` = unlimited (default); any positive value in MB.

10.3 General

Language

FeedDownloader Pro is available in 2 languages: **Italiano** and **English**.

The language change is immediate: the interface updates without needing to restart the software. The application uses exclusively the dark "Obsidian Command" theme: no light theme or list density selector is available.

Automatic Feed Refresh

Allows all feeds to be automatically synchronised at regular intervals, without manual intervention. A check also runs at every application startup and when the connection returns after a period offline. Four presets are available:

Option	Behaviour
Disabled	No periodic synchronisation (the startup check remains).
6 hours (default)	Full synchronisation every 6 hours.
12 hours	Full synchronisation every 12 hours.
24 hours	Full synchronisation every 24 hours.

The change is immediate and does not require restarting the software. If new episodes are found during the automatic synchronisation, a clickable operating system notification is sent: it brings the app to the foreground and, if it concerns a single podcast, opens that feed. Automatic synchronisation does not start downloads: it only signals the availability of new content. For a detailed description of the behaviour, see section 5.9.

User Guide and What's New

Also in the **General** tab you will find:

- **"User Guide"**: Opens the built-in documentation, with the **"Open the full PDF manual"** button to consult this manual.

- **"What's new in this version":** Opens the in-app changelog with the release notes of the installed version. The same window appears automatically on the first launch after an update.

Note: The episode list filter bar (text search, status, date, duration) is documented in section 3.4, as it is part of the episode list rather than a setting.

10.4 Security: The Multi-Level Anti-SSRF System

This section is documented for informational purposes: the security system operates in a completely automatic manner and does not require any configuration from the user.

What is an SSRF attack? SSRF (Server-Side Request Forgery) is a type of attack in which a malicious URL, instead of pointing to a public resource, points to internal network resources (such as the router administration panel, a NAS, or a local server). In the context of a downloader, a carefully crafted RSS feed could include audio URLs pointing to these internal resources.

The validation levels:

1. **URL syntactic validation:** The URL is analysed to verify conformance with the standard.
2. **Protocol validation:** Only `http://` and `https://` protocols are accepted. Protocols such as `file://`, `ftp://`, `data:`, `javascript:` are rejected immediately.

3. **Blocking of well-known internal hostnames:** Names such as `local-host` and literal loopback addresses are rejected.
4. **Blocking of private and reserved IP addresses:** All IP addresses belonging to private or reserved ranges are blocked, including:
 - `10.0.0.0/8` , `172.16.0.0/12` , `192.168.0.0/16` (RFC 1918 private networks)
 - `127.0.0.0/8` (loopback)
 - `169.254.0.0/16` (link-local)
 - `::1/128` (IPv6 loopback)
 - `fc00::/7` (IPv6 unique local)
 - Any address pointing to the local host.
5. **Re-validation at connection time:** The DNS-resolved IP address is checked again on every connection and every redirect. This neutralises DNS-based evasion techniques (a public domain resolving to a private address, or a redirect towards an internal resource).

Note for corporate environments: If the corporate network includes internal podcast servers reachable via private IP addresses, the anti-SSRF system will block these URLs. In this case, contact support for a custom configuration that includes specific IP address ranges in the internal whitelist.

10.5 Advanced

Updates

FeedDownloader Pro includes an integrated, **consent-based** update system: no download and no installation ever happen automatically.

Automatic check on startup: In the installed version (package), the software automatically checks for the availability of new updates shortly after startup, by querying the releases repository. If a new version is available, the persistent **"Update available"** indicator appears in the top bar, along with a system notification — but **nothing is downloaded**.

Manual check: The **"Check for Updates"** button in the **Advanced** tab forces an immediate check at any time.

Consent-based download: The package download starts only when you press **"Download"** in the top-bar indicator, or **"Download update"** in the Settings. Once the download completes, the **"Restart & Install"** button appears: installation, too, requires an explicit user action.

States displayed during the process:

- **Checking for updates...** — the software is querying the releases repository.
- **You have the latest version** — the installed version is the most recent.
- **Update available: vX.Y.Z** — waiting for download consent.
- **Downloading update... N%** — download in progress (started by the user).
- **Update downloaded — restart to install** — the package is ready for installation.

Maintenance

The "**Clean temporary files**" button removes orphaned `.part` files left in the destination folder by past interrupted downloads. The function is unavailable while downloads are running (their `.part` files are in use). On completion, the number of removed files is shown.

Danger Zone: Reset Download History

The "**Reset Download History**" button clears the download history recorded in the database (the "tick marks" of downloaded episodes). The software requires explicit confirmation before proceeding. Audio files on disk are **not** deleted: after the reset, episodes will appear as not downloaded again.

When to use it: Only when you intend to start from an empty history, for example after migrating to a new system or to remove data from a test cycle.

Go to Chapter 11 for troubleshooting the most common problems.

Chapter 11: Troubleshooting

11.1 How to Use This Chapter

This chapter collects the most common problems reported by users, with the most likely causes and step-by-step solutions. Each problem is described in the way it manifests in the interface, not in internal technical terms.

If the problem is not in this list, contact support describing the steps that reproduce it and the error message displayed.

Feed and Analysis Problems

Problem: "Connection error" or "Timeout" during feed analysis

How it manifests: You click **"Analyse"** and after a few seconds an error message appears indicating a timeout or connection failure. The list remains empty.

Likely causes and solutions:

- **The feed server is unavailable.** Open the feed URL in the browser. If the browser returns an error (page not found, "This site can't be

reached"), the problem lies with the podcast server: nothing can be done except retrying later.

- **The internet connection is unavailable or unstable.** Verify that other websites are reachable. If the connection is unstable, wait for it to stabilise before retrying.
 - **A corporate firewall or proxy is blocking the request.** In corporate environments, traffic to certain hosts may be blocked. Try from a home network to verify whether the problem is specific to the corporate network.
-

Problem: The feed is analysed but the episode list is empty

How it manifests: The analysis completes without errors, but the episode list shows no items (or shows 0 episodes).

Likely causes and solutions:

- **The feed contains no episodes.** Open the URL in the browser and verify that the XML document contains `<item>` or `<entry>` tags. If they are not present, the podcast has not yet published any episodes.
 - **The feed uses a non-standard format.** FeedDownloader Pro supports RSS 2.0 and Atom 1.0. Some feeds produced by proprietary platforms may have an unconventional structure. In this case, the software displays a specific warning in the analysis message.
 - **All episodes are already in the database.** If the feed has been analysed previously, episodes appear with the "ARCHIVED" tag. Scroll through the list and check for this indicator, or use the "Downloaded" status filter.
-

Problem: The feed shows only the last N episodes and not the full historical catalogue

How it manifests: You analyse a podcast with hundreds of known episodes, but the list shows only 50 or 100.

Cause: This limit is imposed by the podcast publisher or its hosting platform, not by FeedDownloader Pro. Many platforms limit the RSS feed to the last 50–100 episodes to reduce the load on their servers.

What the software already does: If the platform publishes the historical archive in multiple linked pages following the **RFC 5005** standard (`rel="next"` links), FeedDownloader Pro follows them automatically and reassembles the entire catalogue. The limit only arises when the feed offers no pagination at all.

Possible alternatives:

- Check whether the podcast offers a "full feed" as an alternative URL (some platforms make this available).
- Consult the podcast's website or distribution platform (Spotify, Apple Podcasts) to retrieve links for older episodes.
- Some platforms accept parameters in the URL to request the complete feed (e.g. `?limit=0` or `?paged=all`): check the documentation of the specific platform.

Download Problems

Problem: Many episodes show "Error 404" status

How it manifests: After a batch download, numerous episodes show "Error" status with the message `404 Not Found`.

Cause: The episodes are still present in the RSS feed (in the XML document), but the audio files they point to have been removed from the server. This situation is common with abandoned podcasts or those migrated to other platforms.

What can be done:

- It is not possible to download files that no longer exist on the server.
 - If this is an active podcast and the errors seem excessive, contact the podcast publisher: it may be a temporary migration or a resolvable technical issue.
 - Episodes with 404 errors are automatically excluded from subsequent batches. It is not necessary to remove them from the list.
-

Problem: Downloads start but proceed very slowly

How it manifests: The progress bar moves, but the speed is very low (a few KB/s) relative to the available bandwidth.

Likely causes and solutions:

- **The podcast server applies bandwidth limitations.** Many hosting servers impose throttling to contain costs. Reducing threads to 1 may improve the situation with servers that penalise multiple connections.
- **The Wi-Fi connection is unstable.** For intensive batch downloads, use a wired (Ethernet) connection.

- **The destination drive is slow.** Writing to a NAS over a Wi-Fi connection or to USB 2.0 devices can be the bottleneck. Consider downloading to a fast local drive first.
 - **The internet connection is genuinely limited.** Check the actual download speed with a speed test. If the result is below expectations, the problem lies with the connection.
-

Problem: An episode remains stuck at a high percentage and never completes

How it manifests: A single download shows a high percentage (90%, 95%, 99%) that never reaches 100% and does not update.

Cause: The server sent almost all of the file but interrupted the transfer before completion. The stall detection will detect this condition within 60 seconds of the last data received and will automatically restart the download.

If the problem persists after multiple attempts: The file on the server may be corrupted or truncated. After the maximum number of attempts, the episode will be marked as **"Error"** with a message indicating a discrepancy between the declared size and the received size.

Problem: The software downloaded an `.mp3` file but the audio player reports it as corrupted

How it manifests: The download shows as completed (green status), but opening the file with an audio player returns an error or the file does not play.

Cause: This should not occur thanks to the `.part` file mechanism and size verification. If it does happen, the original file on the server may already be corrupted (a publisher issue), or a disk write error occurred.

Solution:

1. Hover over the episode in the list and click **"Re-download"** (also available in the Detail Panel).
2. If the re-downloaded file is still corrupted, the problem lies with the source file on the podcast server. Verify this by opening the file URL directly in the browser.
3. Run a Health Check (see Chapter 9): the SHA-256 verification flags every file in the archive whose content no longer matches what was recorded at download time.

Note: Since v1.5.0 the software also rejects non-audio enclosures: if the server sends a web page instead of the file, the download fails with the message *"The server sent a web page, not audio"* instead of saving an unusable file.

NAS and Network Problems

Problem: "Network path not reachable" even though the NAS is switched on

How it manifests: The software shows the path not reachable warning, but the NAS is accessible normally from the file manager.

Solutions to check in order:

1. **Verify that the path is exact.** A difference in capitalisation (`\\MYNAS\podcast` vs `\\MYNAS\Podcast`) can cause an error on some systems.
 2. **Are the SMB credentials stored?** Open File Explorer and attempt to access `\\MYNAS\ShareName` manually. If a password is requested, the credentials are not saved in the Windows Credential Manager. Enter them and tick **"Remember"**.
 3. **Is the Windows Firewall blocking FeedDownloader Pro?** Go to `Control Panel → Windows Defender Firewall → Allowed apps` and verify that FeedDownloader Pro is listed with access permitted.
 4. **Does the NAS support SMBv2/3?** Some older NAS devices support only SMBv1, which is disabled by default on Windows 11. Update the NAS firmware or enable SMBv1 from the NAS administration panel.
-

Problem: Downloads to NAS stop after a few minutes

How it manifests: The batch starts normally, downloads a few episodes, then stalls with write errors or a path not reachable error.

Cause: The NAS enters sleep mode during the download. Some consumer NAS devices have a power-saving function that can activate even during active transfers if the device is configured to monitor only web traffic, ignoring SMB connections.

Solutions:

- Temporarily disable sleep mode from the NAS administration panel during batch downloads.

- Reduce the number of threads to 1: a continuous write stream prevents sleep activation more effectively than intensive bursts with intermediate pauses.
-

General Problems

Problem: The interface responds with a delay

How it manifests: Clicks take 1–2 seconds to respond, scrolling through the list is jerky, the programme appears slow.

Likely causes:

- **Large database.** With tens of thousands of episodes in the database, some operations may slow down. Consider using "**Reset Download History**" (**Settings** → **Advanced**) only if the archive contains many episodes in error status or data that you do not intend to recover.
 - **High number of threads on hardware with limited RAM.** With 5 active threads on a system with less than 4 GB of RAM, the process may be slow. Reduce threads to 1 or 3.
 - **Antivirus scanning `.part` files in real time.** Some security software intercepts every disk write operation, slowing downloads. Add the destination folder to the antivirus exclusions.
-

Problem: The software does not start or closes immediately upon opening

How it manifests: The programme is started, the process briefly appears in the Task Manager but then disappears without the interface being displayed.

Solutions:

1. **Corrupted database — automatic recovery.** If the `feeddownloader.sqlite` file turns out to be corrupted at startup, the software automatically sets it aside by renaming it `feeddownloader.sqlite.corrupt-[date]` and starts with a fresh database: the application launches anyway.
 2. **Guided restore.** On the first launch after a recovery (current database empty and a `.corrupt-*` backup present), the software shows the dialogue "A backup of a damaged database was found." with the "Attempt restore" and "Ignore" buttons. Choosing to restore recovers feeds, archive and history from the backup as far as possible; audio files are untouched and the backup is not modified. A summary then reports how many feeds and episodes were recovered.
 3. **Reinstall the software.** Uninstall FeedDownloader Pro and install the most recent version. The database and settings are not deleted by the uninstallation.
-

Problem: I have lost my database data — is it possible to recover it?

How it manifests: The database has been accidentally deleted, is corrupted, or a reset was performed without a prior backup.

Recovery possibilities:

- **With a backup available:** Copy the backup `feeddownloader.sqlite` file to the application's user data folder, with the programme closed (see Chapter 2 for the user data folder path).
- **After a corruption:** Check for `feeddownloader.sqlite.corrupt-[date]` files in the user data folder: if the current database is empty, the software automatically offers the guided restore at startup (see the previous problem).
- **Without a backup:** The audio files on disk are still present: only the software's memory has been lost. The "**Repair archive (checksum search)**" function cannot help here (the hashes were in the lost database), but missing episodes can be re-downloaded or re-catalogued by analysing the feeds again.
- **Prevention:** Periodically make a manual copy of the `feeddownloader.sqlite` file to a safe location, or export the feed list in OPML format (see Chapter 5) as a configuration backup. It is advisable to perform this backup before any migration or software update.

This is the final chapter of the Runtime FeedDownloader Pro Advanced User Manual.

For assistance not covered by this manual, refer to the official releases page or contact the technical support of Ecosystem Runtime | Digital Core.

Ecosystem Runtime | Digital Core — Tools built to last.